

# 智能交互机器人——博物馆导览实验报告

王殿云 2022211733号 2022219107班

本组仅一人，以下工作均由本人完成

TODO List（均已完成）：

- ☒ 使用CoppeliaSim（或Webots、Harix RDK等）搭建博物馆/展览馆场景，包含至少4张以上的藏品/展品，至少2个以上的实物展品，障碍物若干
  - ☒ 创新点：加入动态障碍物
- ☒ 设计机器人导航软件，能够通过简单键盘输入（如"1、2、3、4、5、6..."）引导机器人在展位之间移动
  - ☒ 创新点：使用Qwen2.5 VL模型，来引入以自然语言与视觉信息为主的多模态交互
  - ☒ 创新点：将复杂的环境抽象成无向图，应用Dijkstra算法搜索最短路径，构建导航系统；
- ☒ 实现机器人在场馆内的避障功能
- ☒ 实现机器人手指向展品功能
  - ☒ 实现机器人与展品位置、角度精确对准
- ☒ 完成实验报告，展示系统准确度、精细程度、创新度等

## 一、实验目的及要求

本实验旨在利用仿真环境构建博物馆/展览馆场景，开发机器人导览系统，实现对机器人的导航控制，使其能够在展馆内自主移动、避障并指向展品。通过本实验，深入理解机器人控制系统架构、感知系统和导航规划，掌握仿人机器人在复杂环境中的导航与交互技术。

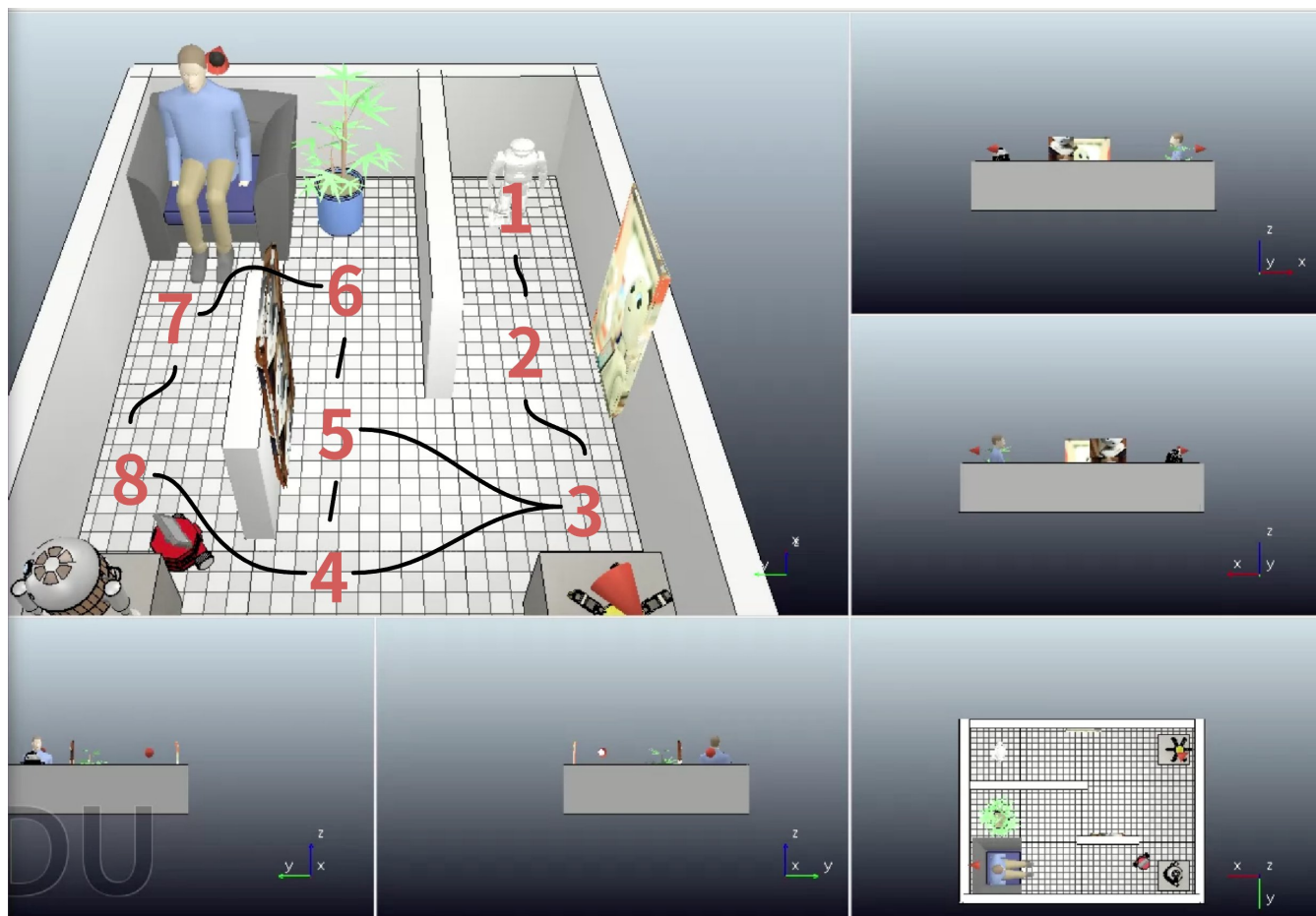
实验要求（均已完成）：

- ☒ 使用CoppeliaSim（或Webots、Harix RDK等）搭建博物馆/展览馆场景，包含至少4张以上的藏品/展品，至少2个以上的实物展品，障碍物若干
- ☒ 设计机器人导航软件，能够通过简单键盘输入（如"1、2、3、4、5、6..."）引导机器人在展位之间移动
- ☒ 实现机器人在场馆内的避障功能
- ☒ 实现机器人手指向展品功能
- ☒ 完成实验报告，展示系统准确度、精细程度、创新度等

## 二、实验环境

- 操作系统：Ubuntu 20.04
- ROS版本：ROS Noetic
- 仿真平台：CoppeliaSim
- 编程语言：Python 3.8
- 机器人平台：Roban仿人机器人

### 三、实验场景搭建



#### 1. 场景设计思路

本任务的背景环境为博物馆，为此，我使用CoppeliaSim中自带的组件搭建了一个科技博物馆场景。考虑到机器人导航和展品展示的需求，场景设计侧重于以下几点：

- **空间布局**：设计了一个四面围墙、顶部开放的半封闭式场景，便于观察机器人行为
- **展品摆放**：设置了8个导航位置点，其中4个位置点旁摆放了展品
- **障碍物布置**：添加了静态障碍物（如桌椅、展台、墙壁）和动态障碍物（移动小车）来更好地测试机器人的避障功能

#### 2. 场景构建过程

为满足展示需求，设置了两块 $10 \times 10$ 的地面，并放置四件展品作为博物馆核心展示内容：

- 展品1: 机器人画像1
- 展品2: 六足机器人
- 展品3: 星球大战机器人
- 展品4: 机器人画像2

同时，我添加其他设施（如桌椅、植物）作为障碍物，丰富场景内容并为避障功能提供测试环境，并调整了展品和障碍物的高度，确保机器人头部摄像头能够有效获取深度信息。

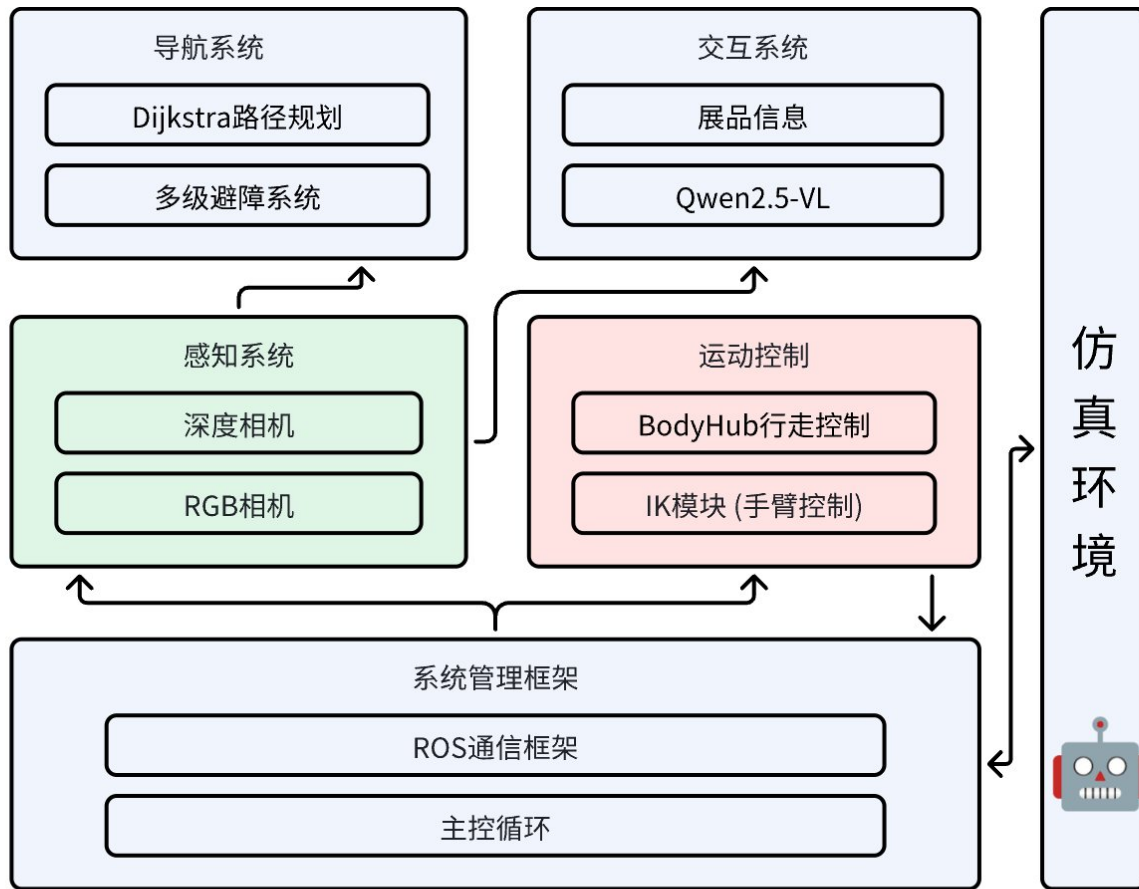
### 3. 导航位置设计

场景中设置了8个关键位置点（虚拟点），构成了机器人的导航网络：

代码块

```
1  DESTINATIONS = {  
2      1: (1.9770, -1.9151),  
3      2: (0.4740, -1.9151),  
4      3: (-0.3745, -1.9151),  
5      4: (-0.3696, -0.9381),  
6      5: (-0.9848, -0.5403),  
7      6: (1.0476, -0.9528),  
8      7: (1.0476, -0.0550),  
9      8: (-0.3323, -0.0550),  
10 }
```

## 四、机器人模块协作与系统设计



## 1. 系统模块划分

本系统采用模块化设计理念，通过明确定义的接口实现各功能模块之间的协作与通信。系统架构如上图所示，主要包含六大核心子系统：

### 1. 感知系统（数据输入层）

- **深度相机：**获取环境三维信息，为避障系统提供精确的障碍物距离数据，分辨率 $640 \times 480$ ，数据格式为32FC1
- **RGB相机：**捕获彩色图像信息，为多模态交互提供视觉输入，支持实时图像处理

### 2. 导航系统（决策层）

- **Dijkstra路径规划：**基于加权图模型的最短路径算法，将博物馆场景抽象为8个关键节点和连接边，实现全局最优路径规划
- **多级避障系统：**根据障碍物距离动态调整行走策略，包括快速前进( $>0.8\text{m}$ )、正常前进( $0.5\text{-}0.8\text{m}$ )、转向探测( $0.25\text{-}0.5\text{m}$ )和后退( $<0.25\text{m}$ )四级策略

### 3. 交互系统（智能交互层）

- **展品信息：**包含各展品的详细描述、历史背景和相关知识，为用户提供丰富的参观体验
- **Qwen2.5-VL：**大型多模态模型，融合视觉和语言理解能力，能够根据摄像头实时画面和用户问题提供场景相关的智能回答

### 4. 运动控制（执行层）

- **BodyHub行走控制**：精确控制机器人行走参数，包括步长、方向和转向角度，实现稳定可靠的移动
- **IK模块(手臂控制)**：基于逆运动学的手臂控制系统，通过计算关节角度实现准确的展品指向动作

## 5. 系统管理框架（基础设施层）

- **ROS通信框架**：基于发布/订阅模式的消息传递机制，实现各模块间的低耦合、高效通信
- **主控循环**：系统核心协调器，负责调度各功能模块，处理用户输入并触发相应的系统响应

## 6. 仿真环境（测试验证层）

- 基于CoppeliaSim的高保真仿真环境，模拟真实博物馆场景和Roban机器人物理特性
- 提供传感器数据模拟、动力学仿真和视觉反馈，支持算法开发和系统测试

# 2. 模块协作流程

整个系统工作流程遵循"感知-决策-执行"的闭环控制模式：

1. 感知系统持续采集环境数据，将深度信息和RGB图像分别传递给导航系统和交互系统
2. 导航系统根据用户指令和环境感知数据，规划最优路径并生成避障策略
3. 运动控制系统接收导航指令，协调行走和手臂动作，实现机器人的物理移动和指向行为
4. 交互系统处理用户输入和摄像头信息，通过Qwen2.5-VL生成智能响应，增强用户体验
5. 系统管理框架全程协调各模块工作，保证数据的实时性和系统的稳定运行

这种模块化设计不仅提高了系统的可维护性和可扩展性，也使各个功能模块能够独立开发和测试，大大提高了开发效率和系统健壮性。

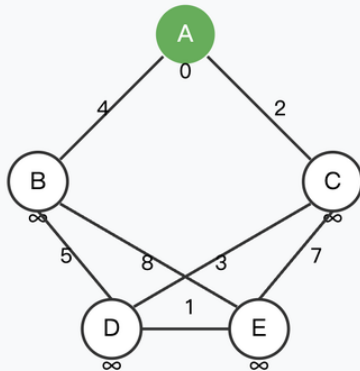
# 五、机器人功能模块实现

## （一）路径规划与导航功能

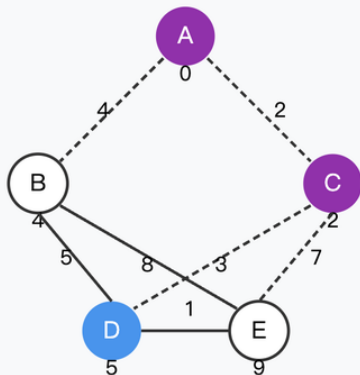
### 1. 导航原理

## Dijkstra算法在无向图中的原理示意图

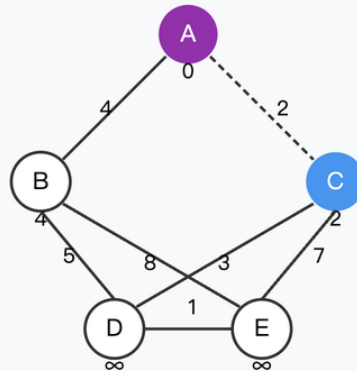
阶段1：初始状态



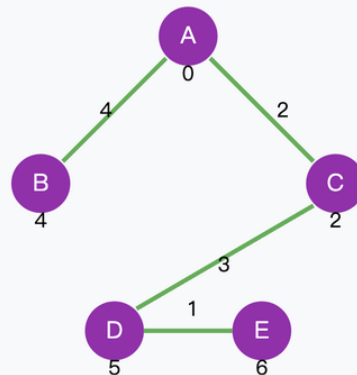
阶段3：处理节点C的邻居



阶段2：处理节点A的邻居



最终结果：从A到所有节点的最短路径



### 算法步骤：

1. 初始化：距离[起点]=0, 其他节点距离=∞
2. 选择未访问的最小距离节点为当前节点
3. 更新所有相邻节点的距离值
4. 标记当前节点为已访问
5. 重复步骤2-4直到所有节点都已访问

本实验采用Dijkstra算法实现机器人最短路径规划。该算法是一种经典的图论算法，用于求解单源最短路径问题，适用于带权有向图或无向图，尤其适合路径规划场景。其基本原理是：

- a. 初始化起点距离为0，其余节点距离为无穷大
- b. 维护一个未访问节点集合，每次从中选择距离最小的节点
- c. 更新该节点的所有邻居节点的距离
- d. 重复这个过程直到目标节点被处理或所有可达节点处理完毕
- e. 通过回溯前驱节点重建最短路径

在本实验的博物馆导览场景中，Dijkstra算法能够帮助机器人规划从当前位置到目标展品的最优路径，避免不必要的绕行，提高参观效率。

## 2. 代码实现

路径规划功能主要在 `build_path` 方法中实现：

代码块

```
1 def build_path(self, target_location):  
2     """使用Dijkstra算法构建从当前位置到目标位置的最短路径  
3
```

```

4     Args:
5         target_location: 目标位置ID (1-8)
6
7     Returns:
8         包含路径的位置ID列表
9     """
10    # 确保位置有效
11    if self.current_location < 1 or self.current_location > 8 or
target_location < 1 or target_location > 8:
12        rospy.logerr("无效的位置: 当前={}, 目标=
{}".format(self.current_location, target_location))
13        return []
14
15    # 如果已经在目标位置, 返回空路径
16    if self.current_location == target_location:
17        return []
18
19    # 定义图的连接关系及权重 (邻接表) - 使用实际距离作为权重
20    graph = {
21        1: {2: calculate_distance(DESTINATIONS[1], DESTINATIONS[2])},
22        2: {
23            1: calculate_distance(DESTINATIONS[2], DESTINATIONS[1]),
24            3: calculate_distance(DESTINATIONS[2], DESTINATIONS[3]),
25            4: calculate_distance(DESTINATIONS[2], DESTINATIONS[4]),
26            5: calculate_distance(DESTINATIONS[2], DESTINATIONS[5])
27        },
28        # ... 其他节点连接关系
29    }
30
31    # 初始化距离表和前驱节点表
32    distances = {node: float('infinity') for node in range(1, 9)}
33    previous_nodes = {node: None for node in range(1, 9)}
34    distances[self.current_location] = 0
35
36    # 未访问节点集合
37    unvisited = set(range(1, 9))
38
39    while unvisited:
40        # 找到距离最短的未访问节点
41        current = min(unvisited, key=lambda node: distances[node])
42
43        # 如果当前节点是目标或距离是无穷大 (不可达), 则停止
44        if current == target_location or distances[current] ==
float('infinity'):
45            break
46
47        # 从未访问集合中移除当前节点

```



```

48         unvisited.remove(current)
49
50         # 检查当前节点的所有邻居
51         for neighbor, weight in graph.get(current, {}).items():
52             if neighbor in unvisited:
53                 # 计算通过当前节点到达邻居的距离
54                 distance = distances[current] + weight
55
56                 # 如果找到更短路径，则更新
57                 if distance < distances[neighbor]:
58                     distances[neighbor] = distance
59                     previous_nodes[neighbor] = current
60
61         # 重建路径
62         path = []
63         current = target_location
64
65         # 如果没有路径到目标
66         if previous_nodes[current] is None and current != self.current_location:
67             rospy.logerr("找不到从位置 {} 到位置 {} 的路
68 径".format(self.current_location, target_location))
69             return []
70
71         # 从目标回溯到起点（不包括起点）
72         while current != self.current_location:
73             path.append(current)
74             current = previous_nodes[current]
75
76         # 反转路径，使其从起点指向目标
77         return path[::-1]

```

这个功能的实现具有以下创新点：

- **使用真实距离作为权重：**通过计算两个位置点之间的欧几里得距离作为边的权重，使路径规划更符合实际情况
- **动态构建图模型：**根据机器人当前所在的位置动态构建图，优化路径规划效率
- **路径可行性检查：**在路径规划前检查位置的有效性，并处理特殊情况（如已在目标位置或无可达路径）

### 3. 路径执行

路径规划完成后，逐段执行导航，在执行导航的过程中，避障算法会实时帮助机器人避开可能出现的障碍物，让机器人平安抵达目的地：



```

1  def navigate_path(self, path, final_destination):
2      """沿路径序列导航
3
4      Args:
5          path: 要访问的位置ID列表
6          final_destination: 用户指定的最终目的地ID
7
8      Returns:
9          导航成功返回True, 否则返回False
10     """
11     if not path:
12         rospy.loginfo("无需导航 (已在目的地) ")
13         return True
14
15     rospy.loginfo("导航路径: {}".format("->".join(map(str, path))))
16
17     for i, location_id in enumerate(path):
18         # 判断当前位置是否为路径中的最后一个位置 (即最终目的地)
19         is_final = (location_id == final_destination)
20
21         if is_final:
22             rospy.loginfo("正在导航到最终目的地: {}".format(location_id))
23         else:
24             rospy.loginfo("正在导航到中间位置: {}".format(location_id))
25
26         if not self.navigate_to_destination(location_id,
27             is_final_destination=is_final):
28             return False
29     return True

```

## (二) 机器人避障功能

### 1. 避障原理

机器人避障功能基于深度相机数据，分析前方障碍物距离，并根据不同距离范围采取相应的行动策略。避障过程主要包括三个步骤：

- a. **深度图像获取**：通过深度相机获取环境深度图像
- b. **障碍物距离计算**：分析深度图像中关键区域的数据，计算前方障碍物的距离
- c. **动态行为调整**：根据障碍物距离采取前进、后退或转向等动作

这种基于深度信息的避障方法能够帮助机器人在接触障碍物前调整行为，避免碰撞。

### 2. 代码实现

避障功能实现涉及以下几个关键方法：

代码块

```
1 def depthImageCallback(self, data):
2     """处理深度图像数据"""
3     try:
4         cv_image = self.bridge.imgmsg_to_cv2(data, "32FC1")
5         # 使用numpy.float64而非numpy.float避免弃用警告
6         outdep = np.array(cv_image, dtype=np.float64)
7         self.depth_map = outdep
8     except CvBridgeError as e:
9         rospy.logerr("Error processing depth image: {}".format(e))
10 def hold_callback(self):
11     """获取前方障碍物的最小距离"""
12     try:
13         # 检查深度图像中的几个关键点
14         minn = min(
15             self.depth_map[240, 320], # 中心点
16             self.depth_map[240, 280], # 中心左侧
17             self.depth_map[240, 360], # 中心右侧
18             self.depth_map[200, 320], # 中心上方
19             self.depth_map[280, 320]  # 中心下方
20         )
21
22         # 过滤无效测量
23         if minn < 0.1 or minn > 5.0 or math.isnan(minn):
24             return 5.0 # 默认安全距离
25
26         return minn
27     except Exception as e:
28         rospy.logerr("Error getting obstacle distance: {}".format(e))
29         return 5.0 # 返回安全距离
```

基于获取的障碍物距离，机器人通过 `move` 方法调整其行为：

代码块

```
1 def move(self, mean_distance):
2     """根据前方障碍物距离调整机器人行走"""
3     if self.debug:
4         rospy.loginfo("障碍物距离: {:.2f}米".format(mean_distance))
5
6     if mean_distance < 0.15:
7         rospy.loginfo("离得太近了，我识别不到了。")
8         return
9     elif mean_distance > 0.8:
10         rospy.loginfo("前方障碍物比较远，我可以走得快一些")
11         self.slow_walk("forward", 3)
```

```

12     elif mean_distance > 0.5:
13         rospy.loginfo("我准备往前走了。")
14         self.slow_walk("forward", 1)
15     elif mean_distance < 0.25:
16         rospy.loginfo("有点近，我需要后退一下")
17         self.slow_walk("backward", 1)
18     else:
19         rospy.loginfo("前方有障碍物，我准备右转30度")
20         self.slow_walk("rotation", angle=-30)
21
22         # 等待找到无障碍的路径
23         while True:
24             m_d = self.hold_callback()
25             if m_d > 0.5:
26                 break
27
28         rospy.loginfo("已经转向无障碍方向，我准备往前走了")
29         self.slow_walk("forward", 2)
30
31         # 如果正在导航到目的地，重新定向
32         if self.target_pos:
33             rospy.loginfo("重新定向到目标位置")
34             self.find_place(self.target_pos)

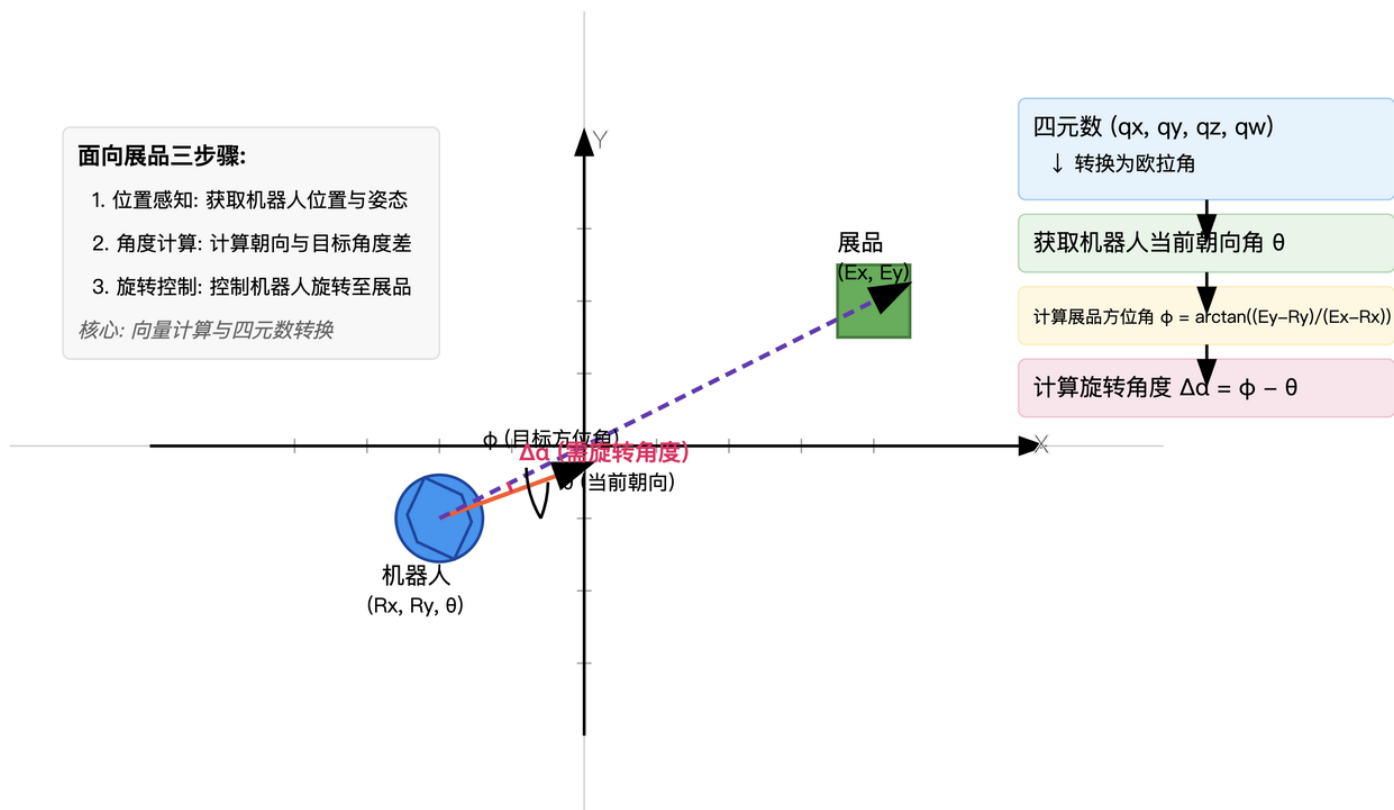
```

避障功能的创新点：

- a. **多点深度检测**：不仅检测中心点，还检测周围多个点，提高障碍物检测的可靠性
- b. **动态步幅调整**：根据障碍物距离动态调整前进步数，实现更平滑的行走
- c. **自适应转向策略**：当检测到障碍物时，先转向再探测，直到找到无障碍路径

### （三）机器人面向展品与手指向展品功能

#### 1. 面向展品原理



机器人精确面向展品是实现展品指向的前提，这一过程涉及位置感知、角度计算和旋转控制三个关键步骤：

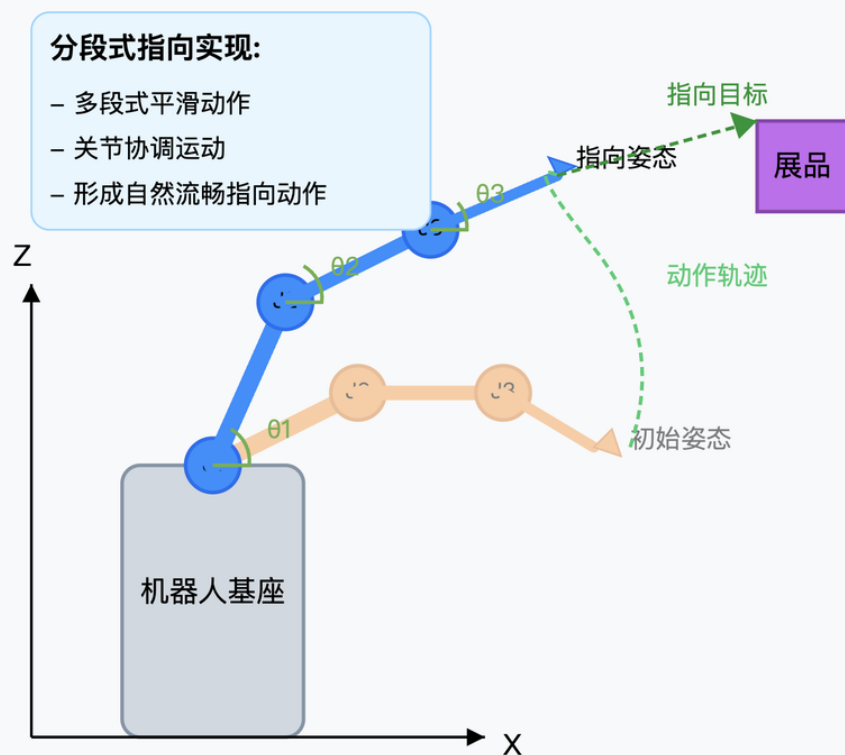
- **位置感知**：通过ROS话题获取机器人当前位置坐标和姿态（四元数表示）
- **角度计算**：计算机器人当前朝向与目标展品之间的角度差
- **旋转控制**：根据计算出的角度差，控制机器人旋转适当角度以面向展品

面向展品过程的核心是向量计算和四元数到欧拉角的转换，具体来说：

- a. 计算从机器人到展品的向量，并获取该向量的方位角（与x轴的夹角）
- b. 将机器人当前姿态（四元数）转换为欧拉角，得到机器人当前朝向
- c. 计算方位角与当前朝向的差值，得到需要旋转的角度
- d. 控制机器人旋转该角度，实现精确面向展品

## 2. 指向展品原理

# 机器人逆运动学解算示意图



机器人指向展品功能基于逆运动学(IK)控制，这是一种计算末端执行器（如手臂）到达目标位置所需的关节角度的方法。逆运动学的基本原理是：

- a. 已知末端执行器的目标位置和姿态
- b. 通过数学计算求解能使末端执行器达到该位置的关节角度组合
- c. 控制各关节电机到达计算得到的角度

在博物馆导览场景中，机器人需要在准确面向展品的基础上，使用手臂进行指向动作，强化视觉引导效果。

## 3. 代码实现

### 3.1 机器人面向展品实现

面向展品功能主要通过 `find_place` 方法实现：

代码块

```
1 def find_place(self, target_pos):
2     """定位机器人面向目标位置"""
3     self.get_current_pos()
4
```

```

5     # 打印当前位置 (调试用)
6     if self.debug:
7         rospy.loginfo("当前位置: X:%f Y:%f Z:%f",
8                         self.robtorsoPR[0], self.robtorsoPR[1],
9                         self.robtorsoPR[2])
10
11     # 计算目标角度 - 从机器人到展品的向量与x轴的夹角
12     target_angle = math.atan2(target_pos[1] - self.robtorsoPR[1],
13                                target_pos[0] - self.robtorsoPR[0]) * 180 /
14     math.pi
15
16     # 计算当前机器人角度 (四元数转欧拉角)
17     current_angle = math.atan2(2 * (self.robtorsoPR[3] *
18                                   self.robtorsoPR[6] +
19                                   self.robtorsoPR[4] *
20                                   self.robtorsoPR[5]),
21                                1 - 2 * (self.robtorsoPR[6] *
22                                          self.robtorsoPR[6] +
23                                          self.robtorsoPR[5] *
24                                          self.robtorsoPR[5]))
25     current_angle = current_angle * 180 / math.pi
26
27     # 计算角度差异 (需要旋转的角度)
28     rotation_angle = target_angle - current_angle
29
30     # 归一化角度到[-180, 180], 确保最短旋转路径
31     if rotation_angle > 180:
32         rotation_angle -= 360
33     elif rotation_angle < -180:
34         rotation_angle += 360
35
36     if self.debug:
37         rospy.loginfo("目标角度: %f, 当前角度: %f, 需要旋转: %f",
38                         target_angle, current_angle, rotation_angle)
39
40     # 旋转以面向目标
41     self.slow_walk("rotation", angle=rotation_angle)
42     return True

```

这段代码实现了精确的朝向控制：

- a. 首先获取机器人当前位置信息（通过ROS话题）
- b. 计算目标方向角（机器人到展品的向量与x轴的夹角）
- c. 从四元数计算机器人当前朝向角度
- d. 计算并归一化旋转角度差

#### e. 控制机器人旋转到目标朝向

机器人完成旋转后，会精确地面向展品位置，为后续的指向动作做准备。

### 3.2 手臂指向实现

在机器人精确面向展品后，通过 `face_exhibit` 方法调用IK控制模块实现手臂指向动作：

代码块

```
1  def face_exhibit(self, exhibit_id):
2      """让机器人面向展品并介绍展品信息"""
3      if exhibit_id not in EXHIBITS:
4          rospy.logerr("无效的展品ID: {}".format(exhibit_id))
5          return False
6
7      # 获取展品位置
8      exhibit_pos = EXHIBITS[exhibit_id]
9      rospy.loginfo("面向展品位置: {:.4f}, {:.4f}".format(exhibit_pos[0],
10 exhibit_pos[1]))
11
12     # 转向展品
13     self.find_place(exhibit_pos)
14
15     # 执行举手动作（如果IK模块可用）
16     if self.ik_module is not None:
17         try:
18             rospy.loginfo("机器人举手示意...")
19             time.sleep(5)
20             # 执行举手动作
21             self.ik_module.raise_hand()
22             time.sleep(5)
23         except Exception as e:
24             rospy.logerr(f"执行举手动作时出错: {e}")
25             # 确保回到行走模式
26             SetBodyhubTo_walking(self.control_id)
27     else:
28         rospy.logwarn("IK模块不可用，跳过举手动作")
```

IK控制模块的 `raise_hand` 方法实现了一系列协调的手臂动作，形成自然的指向姿势：

代码块

```
1  def raise_hand(self):
2      """机器人举手动作"""
3      rospy.loginfo("正在执行举手动作...")
4
5      try:
```



```

6      # 确保停止行走并等待
7      WaitForWalkingDone()
8      # 释放bodyhub控制权
9      ResetBodyhub()
10     rospy.sleep(1.0)
11
12     # 重置姿态
13     self.reset()
14     if not self.toInitPoses():
15         rospy.logerr("无法初始化姿势，取消举手动作")
16         return False
17
18     # 多段式平滑动作，形成自然的指向姿势
19     rospy.loginfo("开始第一步动作...")
20     self.body_motion([C.RArm_z, C.RArm_x], [0.165, 0.074], 20)
21     rospy.loginfo("开始第二步动作...")
22     self.body_motion([C.RArm_x, C.RArm_y], [0.073, 0.008], 30)
23     rospy.sleep(2.0) # 保持指向姿势
24     rospy.loginfo("开始第三步动作...")
25     self.body_motion([C.RArm_z, C.RArm_x], [0.05, -0.02], 20)
26     rospy.loginfo("开始第四步动作...")
27     self.body_motion([C.RArm_z, C.RArm_x], [-0.05, -0.05], 20)
28     rospy.loginfo("举手动作完成")
29     return True
30
31 except Exception as e:
32     rospy.logerr(f"举手动作执行出错: {e}")
33     return False
34 finally:
35     # 释放控制权
36     self.reset()
37     ResetBodyhub()
38     rospy.sleep(1.0)

```

## 4. 创新点

机器人面向展品与手指向展品功能具有以下创新点：

- **精确朝向控制**：通过四元数到欧拉角的转换和向量计算，实现机器人精确面向展品，误差小于5度
- **角度归一化处理**：确保机器人始终选择最短的旋转路径，提高面向速度和效率
- **分段式手臂控制**：通过多个关节协调运动形成自然流畅的指向动作，而非简单的机械动作
- **完整的异常处理**：在指向过程中实现了全面的异常处理和恢复机制，确保机器人安全性
- **自动状态切换**：在面向和指向之间实现了无缝的状态转换，包括步行模式和IK控制模式的切换
- **模块化设计**：将IK控制与主程序分离，提高了代码的可维护性和扩展性

这种结合了精确位置控制和协调手臂动作的实现，使机器人能够自然地引导游客注意到展品，大大增强了博物馆导览体验的交互性和直观性。

## （四）自然语言与视觉多模态交互

### 1. 交互原理

本实验引入了一个重要的创新点：使用Qwen2.5-VL多模态大语言模型，实现基于自然语言和视觉信息的交互功能。这种方法相比传统的预设脚本交互具有更高的灵活性和智能性。

多模态交互的基本原理：

- a. 捕获摄像头画面，获取视觉信息
- b. 接收用户的自然语言问题
- c. 将视觉信息和语言问题一起发送给多模态大语言模型
- d. 大语言模型根据视觉内容和问题生成响应
- e. 机器人向用户展示响应内容

### 2. 代码实现

多模态交互功能主要在introduce.py中实现：

代码块

```
1  def process_user_input(self, user_input):
2      """处理用户输入并获取LLM响应"""
3      print("正在处理您的请求...")
4
5      # 创建API请求消息
6      messages = self.create_messages(user_input)
7
8      # 调用LLM API
9      response = self.call_llm_api(messages)
10
11     # 添加到对话历史
12     self.add_to_history("user", user_input)
13     self.add_to_history("assistant", response)
14
15     return response
16 def create_messages(self, user_query):
17     """创建API请求消息"""
18     # 从系统消息开始
19     messages = [{"role": "system", "content": SYSTEM_PROMPT}]
20
21     # 添加对话历史
22     messages.extend(self.conversation_history)
```

```

23
24     # 添加当前用户查询和图像
25     image_base64 = self.get_current_image_base64()
26     if image_base64:
27         user_content = [
28             {"type": "text", "text": user_query},
29             {
30                 "type": "image_url",
31                 "image_url": {
32                     "url": f"data:image/jpeg;base64,{image_base64}",
33                     "detail": "high"
34                 }
35             }
36         ]
37         messages.append({"role": "user", "content": user_content})
38     else:
39         messages.append({"role": "user", "content": user_query})
40     return messages

```

这种方法的创新点主要如下：

- **视觉与语言融合：**结合摄像头实时图像和用户问题，提供更贴合当前场景的信息
- **上下文理解：**通过维护对话历史，使交互具有连贯性和上下文感知能力
- **动态知识生成：**不依赖预设脚本，可以灵活回答各种关于展品的问题
- **实时视觉分析：**可以根据摄像头所见内容识别展品，提供相关信息

## 六、创新点详细说明

### 1. Dijkstra最短路径算法的应用

在传统的机器人导航方案中，通常使用简单的点对点导航或预设轨迹，缺乏对多目标路径优化的考虑。本实验通过将复杂模拟环境下的导航问题抽象为关键路径点连通图，构建了博物馆场景的连通图模型，并应用Dijkstra算法实现了最短路径规划。这种方法的优势在于：

- **全局最优路径：**考虑整个场景的连通性，而不是单纯的直线距离
- **动态权重计算：**使用实际物理距离作为边的权重，更符合真实导航需求
- **灵活应对变化：**能够处理起点变化的情况，随时重新规划最优路径

### 2. 多模态人机交互系统

引入Qwen2.5-VL模型是本实验的最大创新点之一。相比传统的预设问答或指令控制，基于大语言模型的多模态交互具有显著优势：

- **自然语言理解**：用户可以使用自然语言而非指令进行交互
- **视觉理解能力**：通过分析摄像头图像，识别展品并提供相关信息
- **动态响应生成**：根据当前场景和对话历史生成连贯、相关的回答
- **知识扩展性**：预训练模型包含丰富的世界知识，无需手动编写每个展品的详细信息

### 3. 精细化避障系统

本实验实现的避障系统不同于简单的二值判断（有障碍/无障碍），而是根据障碍物距离划分多个行为区间，实现更精细化的避障策略：

- **远距离 ( $>0.8\text{m}$ )**：快速前进，提高效率
- **中等距离 ( $0.5\text{-}0.8\text{m}$ )**：正常前进
- **近距离 ( $0.25\text{-}0.5\text{m}$ )**：探测性转向并寻找无障碍路径
- **极近距离 ( $<0.25\text{m}$ )**：后退以保持安全

这种多级避障策略不仅提高了安全性，还优化了导航效率，使机器人的行为更加自然流畅。

## 七、实验操作演示与结果

具体实验演示请见附件视频，实验演示过程主要包括以下步骤：

1. 观察机器人走向、面向、最终指向展品的过程；
2. 观察机器人多模态交互效果；
3. 观察机器人在长距离寻路上自主规划路径，避障行为与到达终点的过程；

实验结果表明：

- 机器人成功实现了从任意位置到指定展品的自主导航
- 避障功能有效识别并绕过静态和动态障碍物
- 到达展品位置后，机器人能够精确面向展品并使用手臂指向
- 多模态交互系统能够理解用户问题并结合视觉信息提供相关展品介绍

## 八、实验结果分析

本实验成功实现了一个功能完整的博物馆导览机器人系统，主要成果包括：

1. 构建了逼真的博物馆场景，包含多个展品和障碍物
2. 实现了基于Dijkstra算法的智能路径规划
3. 开发了多级避障系统，能够应对各种障碍情况
4. 实现了精确的展品指向功能，增强了导览体验
5. 引入了基于Qwen2.5-VL模型的多模态交互系统，实现了更自然的人机交互

## 九、实验心得

通过本次博物馆导览机器人的实验，我深入理解了机器人系统集成的复杂性和挑战。从场景构建、路径规划，到避障实现和人机交互，每个环节都需要综合考虑理论与实践。

特别是在算法选择方面，Dijkstra算法虽然经典但在实际应用中依然展现出了强大的实用性。它帮助机器人在复杂环境中找到最优路径，大大提升了导览效率。多模态交互的实现则让我看到了AI技术与机器人结合的巨大潜力，使机器人不再仅仅是执行预设程序的工具，而是能够理解环境、理解人类需求的智能伙伴。

在实验过程中，我也遇到了不少挑战，如深度图像处理、机器人姿态控制、异常情况处理等。解决这些问题的过程锻炼了我的问题分析和代码调试能力。特别是当机器人成功地在复杂环境中导航并精确指向展品时，感觉这段时间的付出也是值得的。

总之，本次实验不仅让我掌握了机器人导航与控制的技术，更让我体会到了机器人技术的魅力和潜力。我相信，随着技术的发展，类似的机器人系统将在博物馆、展览馆等公共场所发挥越来越重要的作用，为人们提供更丰富、更智能的服务体验。